MODULE 2

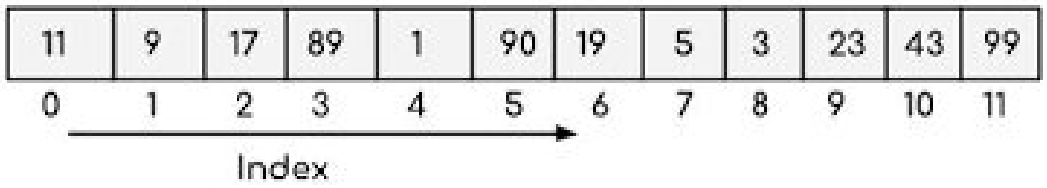
Arrays

Arrays

An array is a fixed size sequential collection of elements of same datatype.

Example

- Marks of students
- List of Univ Roll no.
- Height of soldiers



Arrays

- All arrays consist of contiguous memory locations.
- The lowest address corresponds to the first element and the highest address to the last element.
- There are 2 types of C arrays.

  Single dimensional array

  Multi dimensional array

One dimensional array

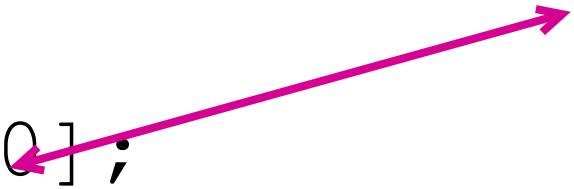
Array - Declaration syntax

```
data_type array_name[size];
```

Example

```
int height[50];
float temperature[30];
char name[20];
double arr[5];
```

(Strings)



Note: Array index
height[0] refers to first height.
height[1] refers to second height.

Array

Array - Initialization syntax

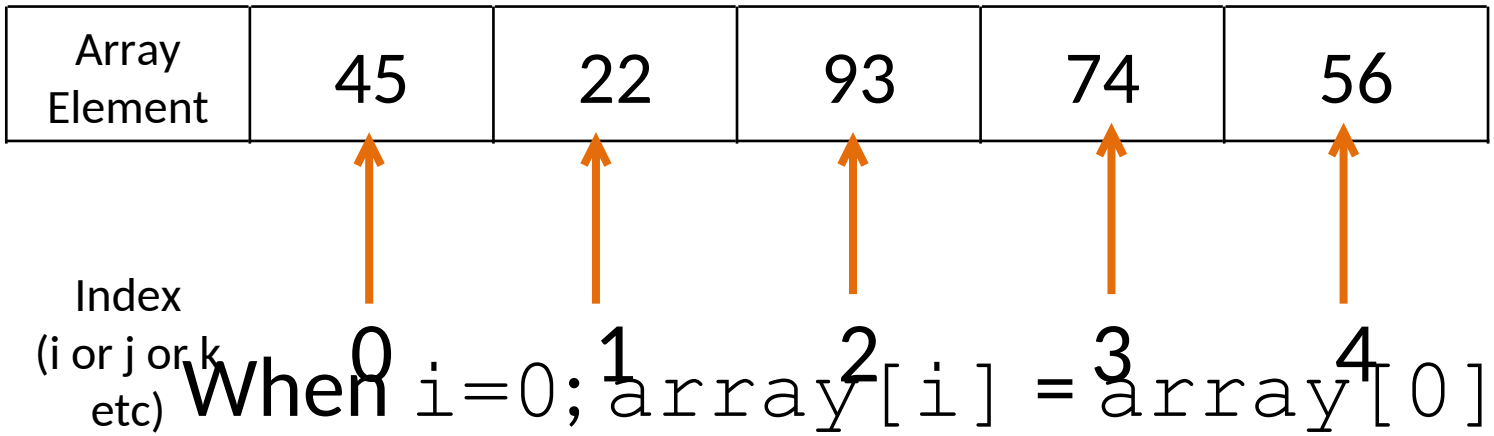
```
data_type array_name[size] = {list of values};
```

Example:

```
int mark[5] = {45,56,98,90,67};
float height[3] = {156.3,172.1,168.35};
```

Basic Programs- array

1. WAP to print 1-D array of size 5
2. WAP to input array of size N and display it



In general, " i "varies from '0' to 'array_size - 1'

WAP to print 1-D array of size 5

```
#include <stdio.h>
int main()
{
   int i;
   int arr[5] = {45,22,93,74,56};
   for (i=0; i<5; i++)
     printf("%d\t",arr[i]);
   return 0;
}
```

Output:

The values of array are :

45 22 93 74 56

WAP to input array of size N and display it

```
#include <stdio.h>
int main()
{
  int i, N, arr[50];
  printf("Enter the size of the array : ");
  scanf("%d", &N);
  printf("Enter the elements of the array: \n");
  for (i=0; i<N; i++)
    scanf("%d", &arr[i]);
  printf("The elements of the array are : \n");
  for (i=0;i<N; i++)
    printf("%d\t",arr[i]);
  return 0;
}
```

Output- array

Enter the size of the array : 3
Enter the elements of the array:
4
50
-6
The elements of the array are :
4 50 -6

WAP to find the average of marks of 5 students

```
#include<stdio.h>
int main()
{
int i, sum, marks[5];
float avg;
sum = 0;
for(i=0;i<5; i++)
  {
  printf("Enter the mark[%d] : ",i);
  scanf("%d", &marks[i]);
  sum = sum + marks[i];
  }
avg =  (float)sum/5;
printf("The average mark is: %f", avg);
return 0;
}
```

Output - average:

Enter the mark[0] : 65
Enter the mark[1] : 23
Enter the mark[2] : 67
Enter the mark[3] : 88
Enter the mark[4] : 96
The average mark is: 67.800003

WAP to find the sum of positive elements in an array of size N

```
#include<stdio.h>
int main()
{
int i, N, sum_pos, a[100];
sum_pos = 0;
printf("Enter the value of N: ");
scanf("%d", &N);
for(i=0; i<N; i++)
  {
  printf("Enter integer a[%d] : ",i);
  scanf("%d", &a[i]);
  if(a[i]>0)
     sum_pos = sum_pos + a[i];
  }
printf("The sum of positive numbers is: %d", sum_pos);
return 0;
}
```

Output-positive numbers

Enter the value of N: 5
Enter integer a[0] : 9
Enter integer a[1] : -8
Enter integer a[2] : 2
Enter integer a[3] : -4
Enter integer a[4] : 6
The sum of positive numbers is: 17

Searching and sorting

Searching	Sorting
Sequential search (Linear search)	Bubble sort
	Selection sort
Binary search	Insertion sort
	Quick sort
	Merge sort
	Shell sort

Algorithm for Linear Search in C

1. Let **key** be the element we are searching for.
2. Check each element in the array by comparing it to the **key.**
3. If any element is equal to the key, return its **index.**
4. If we reach the **end of the array** without finding the element equal to the key, display that the **element is not found.**

WAP to search an element in an array Linear Search

```
#include<stdio.h>
int main()
{
  int i, N, search_num, flag =0; a[100];
  printf("Enter the value of N : ");
  scanf("%d", &N);
  for(i=0; i<N; i++)
  {
    printf("Enter integer a[%d] : ",i);
    scanf("%d", &a[i]);
  }
```

Linear search contd

```
printf("Enter the value of search_num : ");
scanf("%d", &search_num);
for(i=0; i<N; i++)
 {
 if(a[i] == search_num)
  {
   printf("The number %d is present in
                location a[%d]\n", search_num, i);
    flag = 1;
   }
  }
if(flag == 0)
      printf("The number is not found in the array");
  return 0;
}
```

Note: If the number is found you may **break** and exit the loop, after setting the `flag=1;`

Output_1 - Search a number

Enter the value of N : 5
Enter integer a[0] : 20
Enter integer a[1] : 3
Enter integer a[2] : 49
Enter integer a[3] : 37
Enter integer a[4] : 62
Enter the value of search_num : 37
The number 37 is present in location a[3]

Output_2 – Search a number

```
Enter the value of N : 5
Enter integer a[0] : 22
Enter integer a[1] : 10
Enter integer a[2] : 37
Enter integer a[3] : 6
Enter integer a[4] : 37
Enter the value of search_num : 37
The number 37 is present in location a[2]
The number 37 is present in location a[4]
```

Output_3 - Search a number

```
Enter the value of N : 5
Enter integer a[0] : 6
Enter integer a[1] : 7
Enter integer a[2] : 8
Enter integer a[3] : 9
Enter integer a[4] : 10
Enter the value of search_num : 37
The search number 37 is not found in the array
```

Largest of *N* numbers in array

40	50	30	60	70

Store largest value as variable LARGE

Assume a[0] = 40 is LARGE

Compare **LARGE** with all elements

a[0] **Replace LARGE**

40	**50**	30	60	70	**50**
50 ~~40~~	50	**30**	60	70	**50**
50	50	30	**60**	70	**60**
60 ~~50~~	50	30	60	**70**	**70**

The largest element is 70

Largest of *N* numbers in array

```
#include <stdio.h>
int main()
{
  int i, n;
  double arr[100];
  printf("Enter the size of array : ");
  scanf("%d", &n);
  for ( i = 0; i < n; ++i)
  {
    printf("Enter number a[%d]:  ", i);
    scanf("%lf", &arr[i]);
  }
  for (i = 1; i < n; i++) //for storing largest in a[0]
  {
    if (arr[0] < arr[i])
    {
      arr[0] = arr[i];
    }
  }
  printf("The largest element is : %lf", arr[0]);
  return 0;
}
```

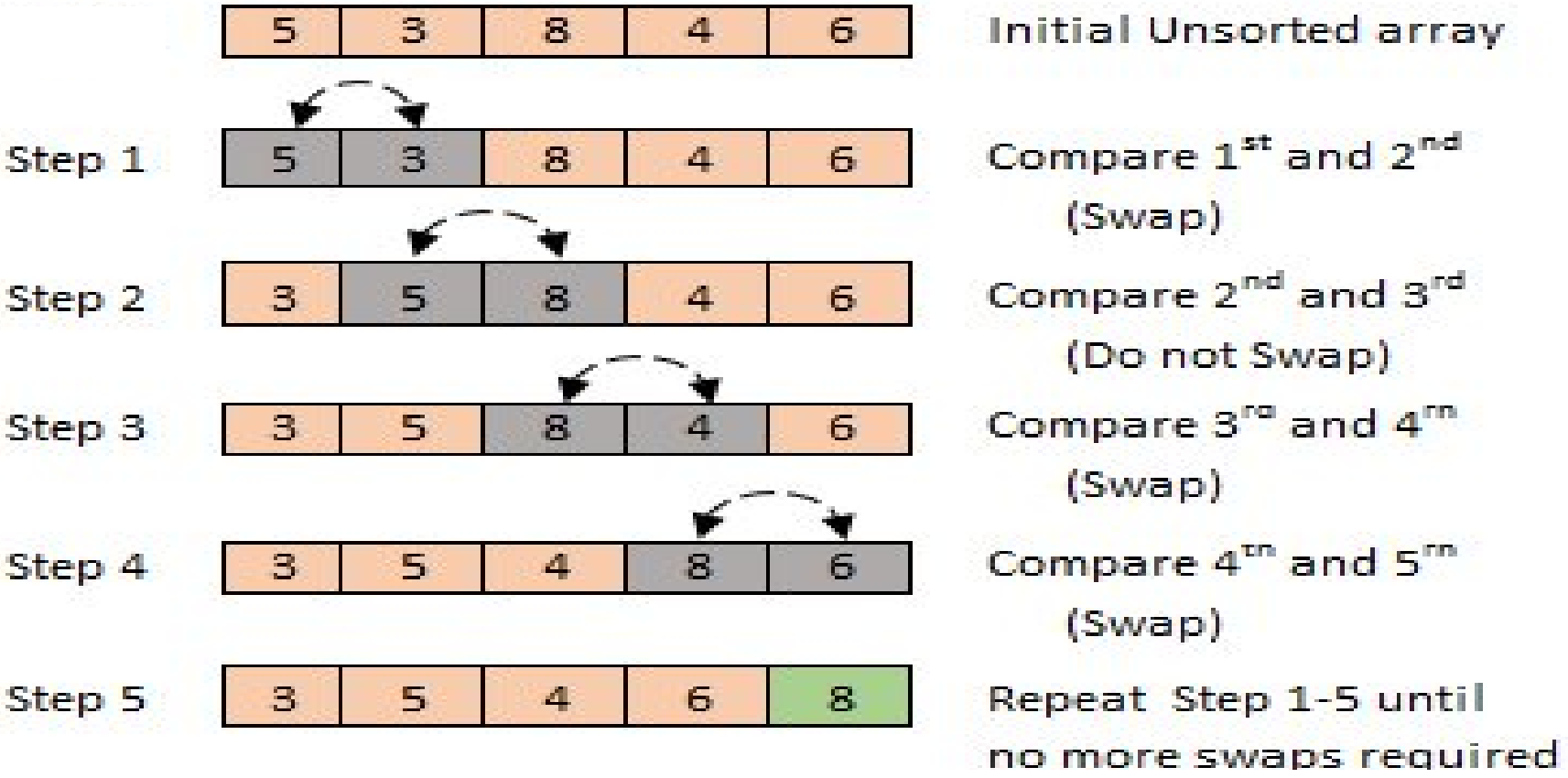
> Smallest
< Largest

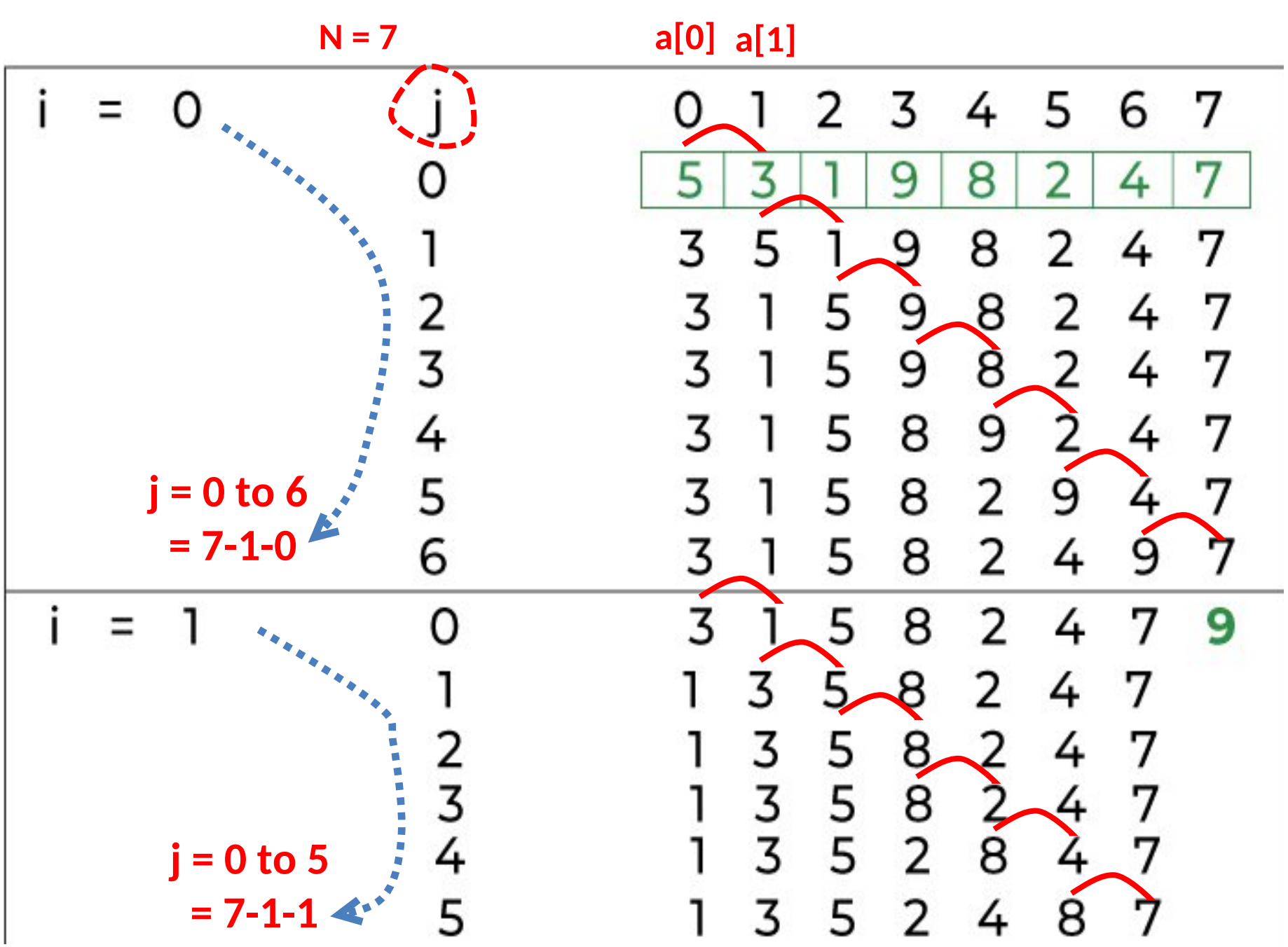
Output – Largest element in array

```
Enter the size of the array : 5
Enter number a[0]:  10
Enter number a[1]:  30
Enter number a[2]:  0
Enter number a[3]:  -45
Enter number a[4]:  15
Largest element  is :  30.000000
```

Bubble sort

- Sorting is the process of arranging numbers in ascending or descending order




N = 7
a[0] a[1]
i = 0
j
0 1 2 3 4 5 6 7
5 3 1 9 8 2 4 7
0
1
2
3
4
5
6
j = 0 to 6
= 7-1-0
3 5 1 9 8 2 4 7
3 1 5 9 8 2 4 7
3 1 5 9 8 2 4 7
3 1 5 8 9 2 4 7
3 1 5 8 2 9 4 7
3 1 5 8 2 4 9 7
i = 1
0
1
2
3
4
5
j = 0 to 5
= 7-1-1
3 1 5 8 2 4 7 9
1 3 5 8 2 4 7
1 3 5 8 2 4 7
1 3 5 8 2 4 7
1 3 5 2 8 4 7
1 3 5 2 4 8 7

i = 2	0	1 3 5 2 4 7	8
	1	1 3 5 2 4 7	
	2	1 3 5 2 4 7	
j = 0 to 4	3	1 3 2 5 4 7	
= 7-1-2	4	1 3 2 4 5 7	
i = 3	0	1 3 2 4 5	7
	1	1 3 2 4 5	
j = 0 to 3	2	1 2 3 4 5	
= 7-1-3	3	1 2 3 4 5	
i = 4	0	1 2 3 4	5
j = 0 to 2	1	1 3 2 4	
= 7-1-i	2	1 2 3 4	
i = 5	0	1 2 3	4
	1	1 2 3	
i = 6	0	1 2	3
		1	2

Sorting - Algorithm

```
loop : i = 0 to n – 1          // outer loop
    loop : j = 0 to n -1- i // inner loop
        if ( a[j] > a[j+1] ) then
            swap a[j] & a[j+1]
    end loop    // inner loop
end loop        // outer loop
```

Swaping for sorting

```
for (i = 0; i < n-1; i++)
{
    for (j = 0; j < (n - 1 - i); j++)
    {
        if (a[j] > a[j + 1])
        {
            temp = a[j];
            a[j] = a[j + 1];
            a[j + 1] = temp;
        }
    }
}
```

Bubble sort (Ascending)

```
#include <stdio.h>
int main()
{
int a[100],i,j,n,temp;
printf("Enter size of array: ");
scanf("%d",&n);
printf("Enter %d elements in the array :\n", n);
for(i=0;i<n;i++)
   {
    scanf("%d",&a[i]);
   }
```

Bubble sort (Ascending) contd

```
printf("\n Elements in array are : ");
for(i=0;i<n;i++)
   {
    printf("%d ",a[i]);
   }
for(i=0; i < n-1; i++)
   {
   for (j=0; j < n-1-i; j++)
     {
      if (a[j] > a[j + 1])
        {
          temp = a[j];
          a[j] = a[j + 1];
          a[j + 1] = temp;
        }
     }
   }
```

> Ascending
< Descending

Bubble sort (Ascending) contd

```
printf("Sorted Elements in array are:");
for(i=0; i < n; i++)
   {
    printf("%d\n ",a[i]);
   }
return 0;
}
```

Output Bubble sort

Enter size of array: 5
Enter 5 elements in the array : 10 20 15 0 45
Elements in array are : 10 20 15 0 45
Sorted Elements in array are : **0 10 15 20 45**

KTU July 2021 Qn:
Write a C program to find the second largest element in an array

WAP to reverse an array

```
#include<stdio.h>
int main()
{
 int i, N, num, temp, arr[100];
 printf("Enter the size of array :");
 scanf("%d", &N);
 printf("Enter the element of the array :\n");
 for (i=0; i<N; i++)
 {
  scanf("%d", &arr[i]);
 }
 printf("The given array is : \n");
 for (i=0; i<N; i++)
 {
  printf("%d\t", arr[i]);
 }
```

WAP to reverse an array

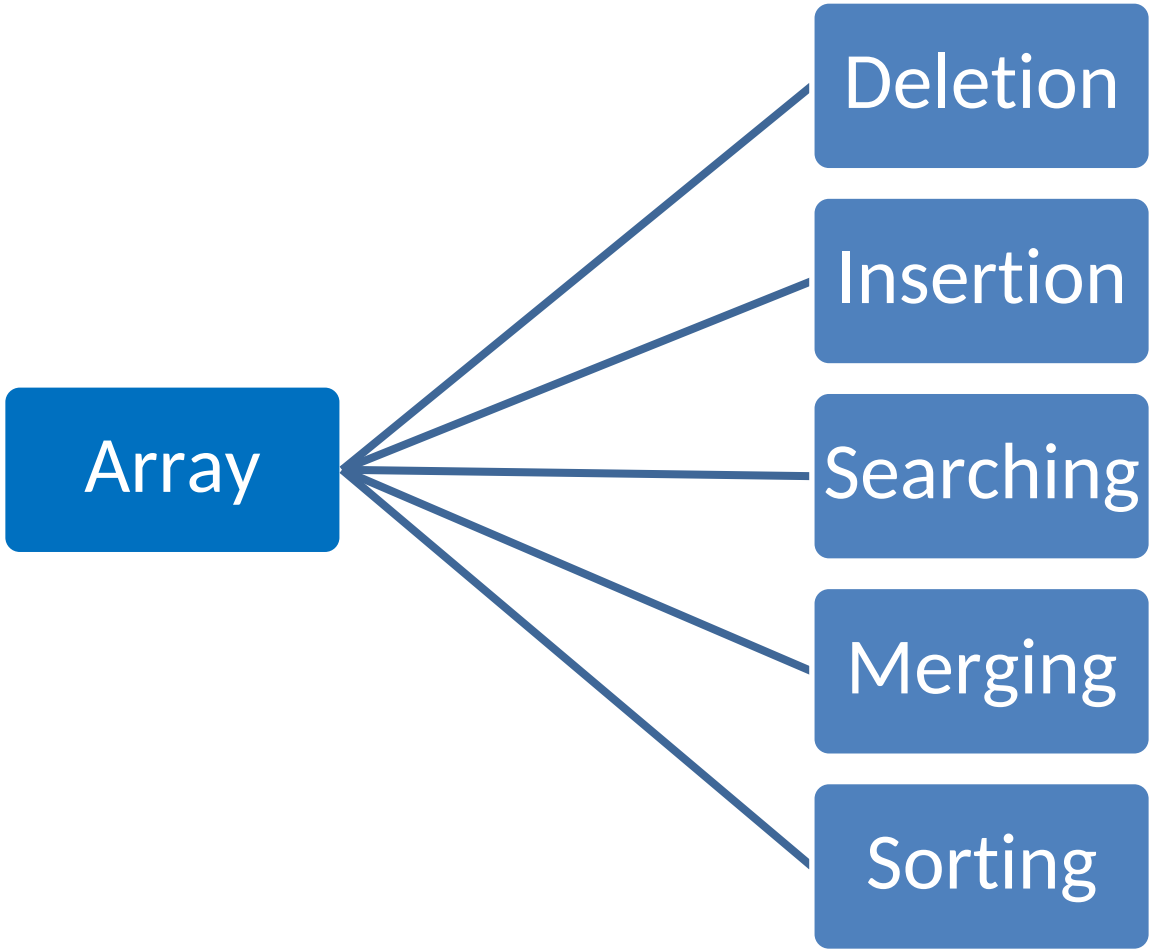
```
for (i=0; i<N/2; i++)
 {
  temp = arr[i];
  arr[i] = arr[N-1-i];
  arr[N-1-i] = temp;
 }
 printf("\nThe array in reverse is : \n");
 for (i=0; i<N; i++)
 {
  printf("%d\t", arr[i]);
 }
 return 0;
}
```

Output

Enter the size of array :5
Enter the element of the array :
1 2 3 4 5
The given array is :
1 2 3 4 5
The array in reverse is :
5 4 3 2 1

KTU July 2024 Model Qn:
Write a C program to reverse the content of an array without using another array.

Operations with Arrays



Programs on 1D Array

- Sum/ Average of elements in an array
- Reverse an array
- Searching an element
  - Find the number of occurrence of an element
- Largest/ Smallest of array elements
- Sorting (Ascending/Descending)
- Merging two arrays
- Insertion of an element

2D Array

2 D - Declaration syntax

datatype array_name[row_size][column_size]

Example

```
int image[100][50];
int matrix[3][4];
float table[30][40];
```

2D Array

2 D - Initialization syntax

```
datatype
array_name[row_size][column_size]=
{ {row1col1, row1col2, ...},
      {row2col1, row2col2, ...},
        ... };
```

Example

```
int disp[2][3] = { {10, 11, 12},
                   {13, 14, 15} };
int disp[2][3] = {10, 11, 12, 13, 14, 15};
int prod[ ][1] = {{3},{4}};
```

Invalid 2-D array initialization

Initialization	Error !!
`int mat[][] = {{1,2},{3,4}};`	array type has incomplete element type
`int sum[2][] = {{2},{1}};`	multidimensional array must have bounds for all dimensions except the first

Write a program to input and display a matrix of size **m x n**

```
#include<stdio.h>
int main()
{
int i, j, m, n;
int a[10][20];

printf("Enter number of rows :  ");

scanf("%d", &m);

printf("Enter number of columns :  ");

scanf("%d", &n);
```

Display a matrix of size **m x n** contd

```
/* Input data in matrix */
for(i = 0; i < m; i++)
{
   for (j = 0; j < n; j++)
   {
   printf("Enter value of a[%d][%d]: ", i, j);
   scanf("%d", &a[i][j]);
   }
}
```

Display a matrix of size **m x n** contd

```
/* Display the matrix */
printf("The matrix is given below :\n");
for (i = 0; i < m; i++)
   {
   for (j = 0; j < n; j++)
      {
        printf("%d\t", a[i][j]);
      }
   printf("\n");
   }
return 0;
}
```

Output – Print_matrix

Enter number of rows : 2

Enter number of columns : 3

Enter the value of a[0][0]: 1
Enter the value of a[0][1]: 2
Enter the value of a[0][2]: 3
Enter the value of a[1][0]: 4
Enter the value of a[1][1]: 5
Enter the value of a[1][2]: 6

The matrix is given below :

1 2 3

4 5 6

Write a program to find Transpose of a square matrix (mxm)

```
#include <stdio.h>
int main()
{
    int i, j, m, a[100][100],  b[100][100] ;
    printf("Enter size of the matrix : ");
    scanf("%d", &m);
    for (i = 0; i < m; i++)
    {
        for (j = 0; j < m; j++)
        {
         printf("Enter the value of a[%d][%d]: ",i,j);
         scanf("%d", &a[i][j]);
        }
    }
```

Transpose of a square matrix (mxm) contd

```
printf("The given matrix is :\n");
for (i = 0; i < m; i++)
   {
     for (j = 0; j < m; j++)
        {
          printf("%d\t", a[i][j]);
        }
        printf("\n");
   }
```

Transpose of a square matrix (mxm) contd

```
for (i = 0; i < m; i++)
  {
  for (j = 0; j < m; j++)
    {
     b[i][j] = a[j][i];
    }
     printf("\n");
  }
```

Transpose operation

```
printf("The Transposed matrix
                        is  :\n");
for (i = 0; i < m; i++)
{
   for (j = 0; j < m; j++)
   {
   printf("%d\t", b[i][j]);
   }
   printf("\n");
}
return 0;
}
```

Output - matrix transpose

```
Enter size of the matrix : 2

Enter the value of a[0][0]: 10
Enter the value of a[0][1]: 25
Enter the value of a[1][0]: 300
Enter the value of a[1][1]: -8
```

The given matrix is :

10	25
300	-8

The Transposed matrix is :

10	300
25	-8

WAP to calculate the sum of two square matrix

```
#include<stdio.h>

int main()

{

int m, a[100][100],b[100][100],sum[100][100], i, j;

printf("Enter the size of matrix : ");

scanf("%d", &m);
```

Sum of two square matrix

Matrix A

```
printf("\n Enter matrix
                    A:\n");
for (i = 0; i < m; ++i)
{
 for (j = 0; j < m; ++j)
 {
 printf("Enter a[%d][%d]:”
                  , i , j );
 scanf("%d", &a[i][j]);
 }
}
```

Matrix B

```
printf("\n Enter matrix
                    B:\n");
for (i = 0; i < m; ++i)
{
 for (j = 0; j < m; ++j)
 {
 printf("Enter b[%d][%d]:”,
                   i , j );
 scanf("%d", &b[i][j]);
 }
}
```

Sum of two square matrix

```
for (i = 0; i < m; ++i)
{
   for (j = 0; j < m; ++j)
   {
   sum[i][j] = a[i][j] + b[i][j];
   }
}
```

Product of Matrix

A (3x3) with index		
00	01	02
10	11	12
20	21	22

B (3x3) with index		
00	01	02
10	11	12
20	21	22

Product c[i][j] - First row

i = 0 j = 0

a[00]*b[00] + a[01]*b[10] + a[02]*b[20]

i = 0 j = 1

a[00]*b[01] + a[01]*b[11] + a[02]*b[21]

i = 0 j = 2

a[00]*b[02] + a[01]*b[12] + a[02]*b[22]

So choose '**k**' varying from 0,1,2

Product of Matrix

```
for (i = 0; i < m; i++)
{
 for (j = 0; j < m; j++)
 {
  c[i][j] = 0;
  for (k = 0; k < m; k++)
  {
   c[i][j]=c[i][j] + a[i][k] * b[k][j];
  }
 }
}
```

Outer loop

Inner loop

Innermost loop

Home work

- WAP to find if sum of all elements in a matrix
- WAP to find if matrix is

  symmetric - $\mathbf{A} = \mathbf{A}^T$

  skew symmetric - $\mathbf{A} = -\mathbf{A}^T$

KTU Qn

1. WAP to display the sum of elements in each row.
2. WAP to check whether a given matrix is a diagonal matrix

WAP to display array elements in reverse

```
#include<stdio.h>
int main()
{
int i, N, a[100];
printf("Enter the value of N : ");
scanf("%d", &N);
for(i=0; i<N; i++)
   {
   printf("Enter the element a[%d] : ",i);
   scanf("%d", &a[i]);
   }
printf(" The array in reverse order is : \n");
for(i=0; i < N; i++)
   {
    printf("%d\n", a[N-1-i]);
   }
return 0;
}
```

Alternate for loop - next slide

WAP to display array elements in reverse - contd

```
#include<stdio.h>
int main()
{
int i, N, a[100];
printf("Enter the value of N : ");
scanf("%d", &N);
for(i=0; i<N; i++)
   {
   printf("Enter the element a[%d] : ",i);
   scanf("%d", &a[i]);
   }
printf("The array in reverse order is : \n");
for(i = N-1; i >= 0; i--)
    {
     printf("%d\n", a[i]);
    }
return 0;
}
```

Output –Array displayed in reverse order

Enter the value of N : 5
Enter the element a[0] : 1
Enter the element a[1] : 2
Enter the element a[2] : 3
Enter the element a[3] : 4
Enter the element a[4] : 5

The array in reverse order is :

5
4
3
2
1